← Back to Unit 2 Index

QUESTION 9 • UNIT 2 – OPTIMIZATION

Explain Gradient Descent. Why does the Vanishing or Exploding Gradients problem happen?

📝 5 Marks

Gradient Descent & Gradient Problems

Optimization algorithm, types, and the vanishing/exploding gradient problem — with diagrams

What is Gradient Descent?

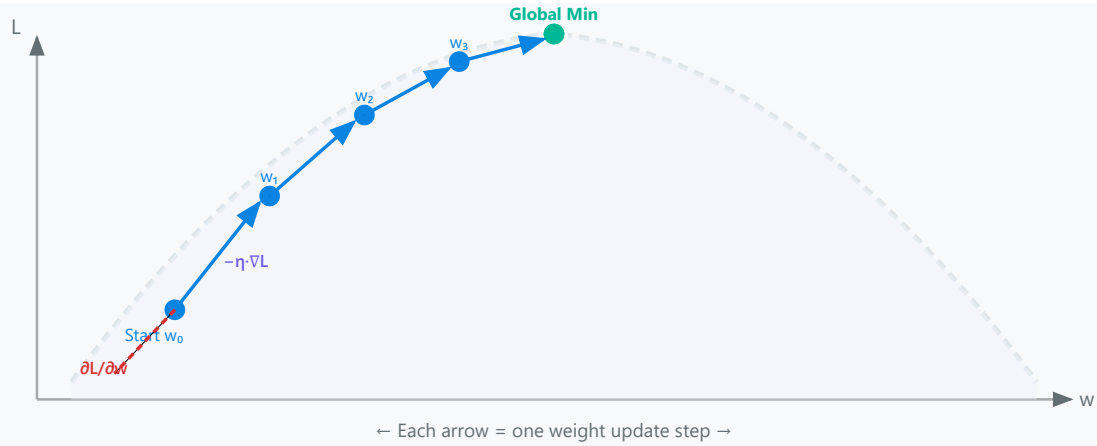
Gradient Descent is an iterative **optimization algorithm** used to minimize the cost (loss) function of a neural network. It adjusts the model's parameters (weights & biases) in the direction of the **steepest descent** of the loss surface.

Update Rule:

$$\text{New Parameter} = \text{Old Parameter} - \text{Learning Rate} \times \text{Gradient}$$

$$w^{(new)} = w^{(old)} - \eta \cdot \frac{\partial L}{\partial w}$$

Figure 1 — Gradient Descent on a Loss Surface (2D Cross-Section)



Gradient descent iteratively moves weights toward the global minimum by following the negative gradient direction, scaled by learning rate η.

How Gradient Descent Works — Steps

1 Initialization

Start with random values for weights w and biases b.

2 Compute Cost Function

Calculate error between predicted output and actual output using a loss function (e.g., MSE or Cross-Entropy).

3 Calculate Gradients

Compute the partial derivative $\partial L/\partial w$ (slope of loss w.r.t. each weight) via backpropagation.

4 Update Parameters

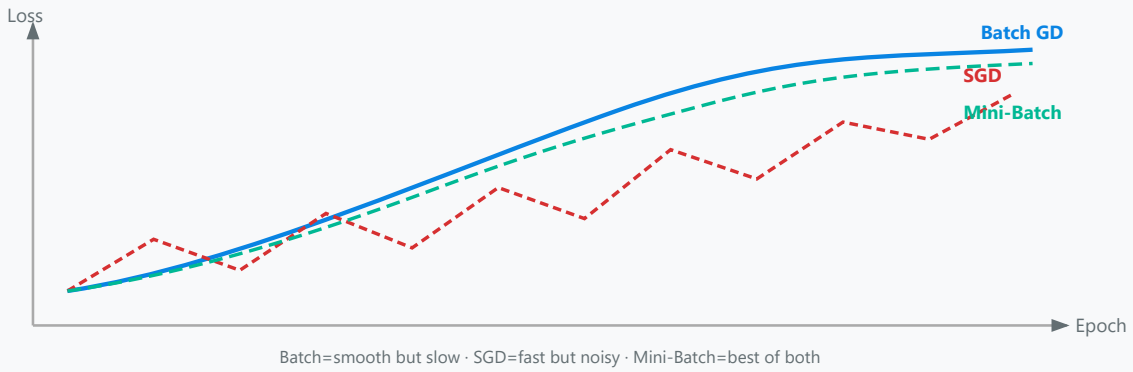
Subtract the scaled gradient from the current weights: $w = w - \eta \cdot \frac{\partial L}{\partial w}$

5 Repeat Until Convergence

Continue steps 2–4 until loss reaches its minimum or a stopping condition is met.

Types of Gradient Descent

Figure 2 — Convergence Paths: Batch vs SGD vs Mini-Batch



Batch GD is smooth but slow; SGD is fast but oscillates; Mini-Batch balances stability and speed.

Batch Gradient Descent

Updates weights after evaluating **all** training samples. Stable but very slow for large datasets.

Stochastic GD (SGD)

Updates weights after **each** sample. Fast but noisy — loss oscillates.

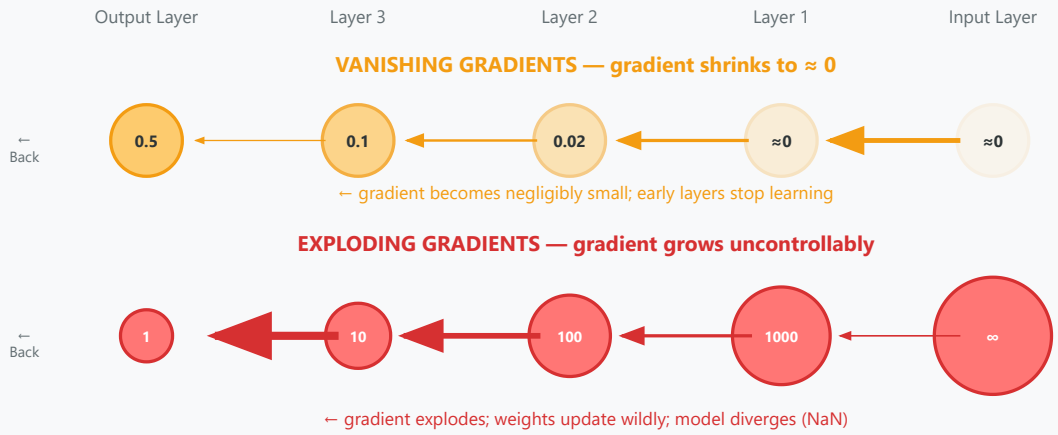
Mini-Batch GD

Updates after a **small batch** of samples. Best of both — efficient and relatively stable.

Vanishing & Exploding Gradients Problem

These problems occur during **backpropagation** in deep networks when gradients are propagated through many layers.

Figure 3 — Gradient Flow: Vanishing vs. Exploding



Top: vanishing gradients cause near-zero updates in early layers. Bottom: exploding gradients cause catastrophic weight updates.

🔒 Vanishing Gradients

What happens: Gradients become very small as they propagate backward through many layers. Early layers receive negligible updates — learning stops.

Why it happens: Activation functions like sigmoid or tanh squash values into small ranges [0,1] or [−1,1]. Their derivatives are small (<0.25), so multiplying many small values across layers drives gradients to ≈ 0.

Impact: Model fails to learn long-range dependencies. Training stalls completely.

💥 Exploding Gradients

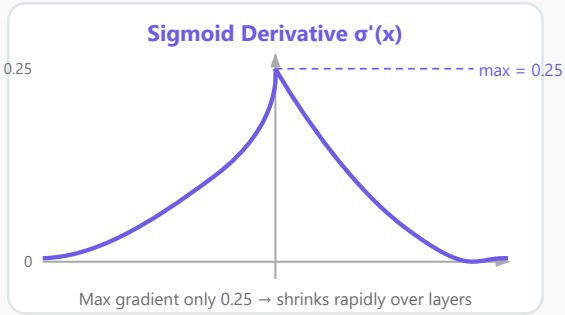
What happens: Gradients become excessively large during backpropagation — weights update by huge amounts, causing the model to diverge (loss = NaN).

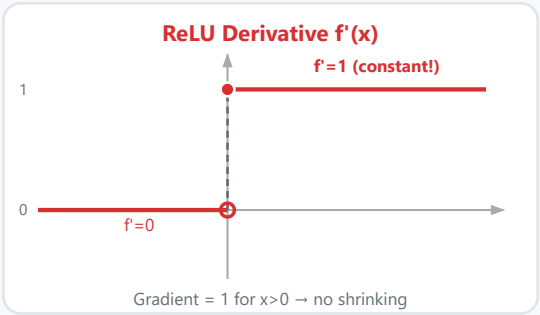
Why it happens: Poor weight initialisation (large initial weights) or very deep networks with weight values > 1, causing gradients to multiply exponentially.

Impact: Model becomes unstable; cost function fails to converge.

Solutions to Vanishing & Exploding Gradients

Figure 4 — Gradient Comparison: Sigmoid vs. ReLU (why ReLU helps)





Sigmoid's max derivative of 0.25 causes gradients to vanish. ReLU's derivative is 1 for positive inputs — gradients pass through unchanged.

Use ReLU Activation

ReLU has derivative 1 for x > 0 — gradients don't shrink. Prevents vanishing gradients in most deep networks.

Batch Normalization

Normalises inputs to each layer to keep activations in a stable range, preventing both vanishing and exploding gradients.

Gradient Clipping

Caps gradient magnitude during backpropagation to prevent them from exploding. Common in RNNs.

Residual Connections (Skip)

Skip connections (ResNet) allow gradients to flow directly between non-adjacent layers, bypassing potential vanishing paths.

LSTM / GRU

Long Short-Term Memory cells maintain constant gradient flow with gating mechanisms — designed to solve vanishing gradients in sequential data.

Proper Weight Initialisation

Xavier (Glorot) or He Initialisation ensures weights start in an appropriate scale, preventing extreme gradient values from the start.